

Week 3: Density Estimation

CSCI 447: Machine Learning– Fall 2026

1

Outlier Detection

- An "outlier" is an example that is "very much different" from other examples in the data set.
- Outliers are not generally regarded as "classes" in the traditional sense of the word.
- Outlier detection is the process of identifying outliers for the purpose of taking separate action (e.g., filtering).
- The process of identifying (i.e., classifying) an example as an outlier leads to so-called "one-class classification."
- Outliers are not generally labeled as such, thus outlier detection is often an "unsupervised learning" problem.

2

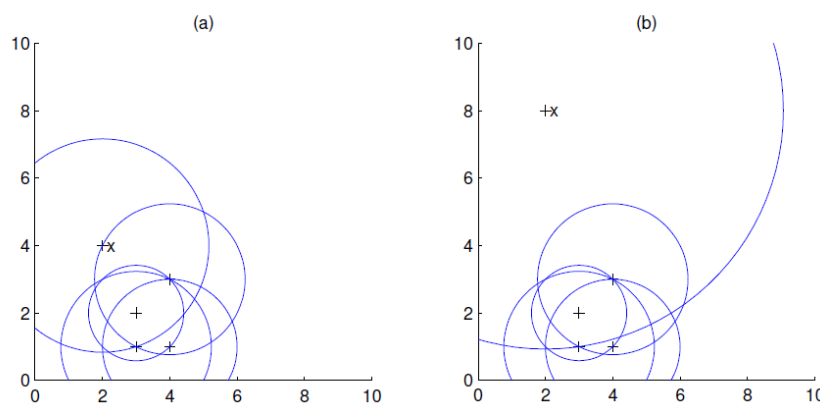
Local Outlier Factor

- Outliers typically occur in sparse areas of the space.
- Compare local density of a data point with average local density of other data points.
- Let $\delta_k(\mathbf{x})$ be the distance between an instance $\mathbf{x}$ and its k th nearest neighbor.
- Let $\mathcal{N}(\mathbf{x})$ be the number of examples in the neighborhood (e.g., the k nearest neighbors) of $\mathbf{x}$.
- For all $\mathbf{s} \in \mathcal{N}(\mathbf{x})$, consider the average $\delta_k(\mathbf{s})$.
- Then

$$LOF(\mathbf{x}) = \frac{\delta_k(\mathbf{x})}{\sum_{\mathbf{s} \in \mathcal{N}(\mathbf{x})} \delta_k(\mathbf{s}) / |\mathcal{N}(\mathbf{x})|}.$$

3

Interpreting LOF



LOF close to 1.0 indicates not an outlier. LOF significantly larger than one indicates an outlier.

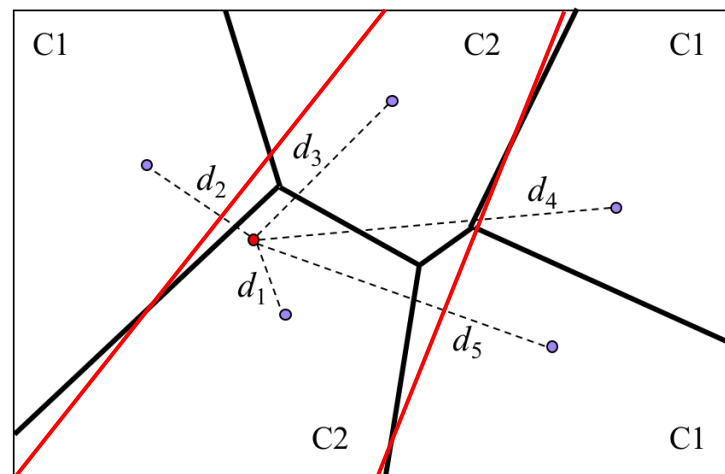
4

Error in Nearest Neighbor

- Generally, it is not reasonable to assume examples accurately represent concepts
- Errors are introduced for several reasons:
 - Noise
 - Lack of precision
 - Human error
 - Irrelevant attributes
- For continuous concepts, examples are at best an “approximation”

5

Error in Nearest Neighbor



6

Irrelevant Attributes

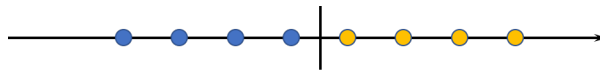


MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

7

Irrelevant Attributes

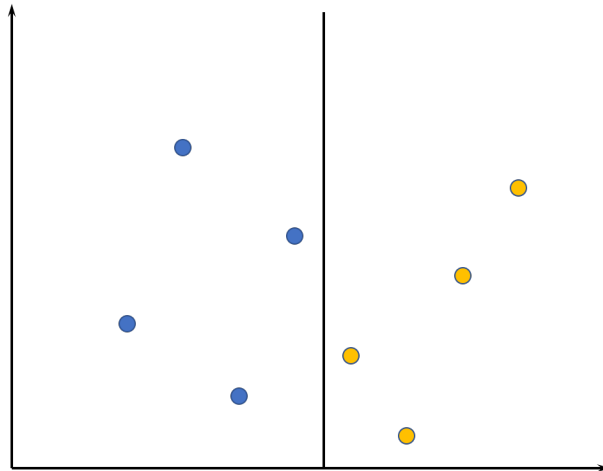


MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

8

Irrelevant Attributes

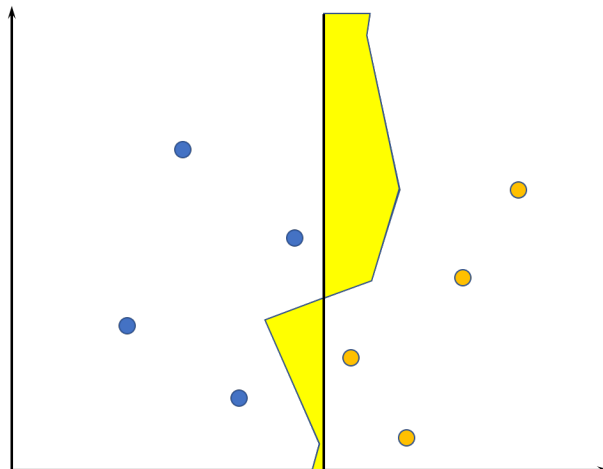


MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

9

Irrelevant Attributes



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

10

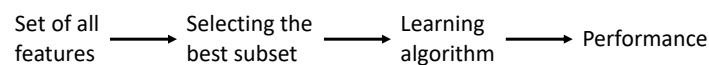
Dimensionality Reduction

- We have two principal categories:
 - Feature selection: Choosing a subset of features
 - Feature extraction: Transform input features to a new feature space
- Example transformation: The Fourier Transform
 - Data is often provided in the “time” domain.
 - Data is converted to the “frequency” domain.
 - Example algorithms: Discrete Fourier Transform and Fast Fourier Transform.
- There is a growing community on what is called “representation learning.”

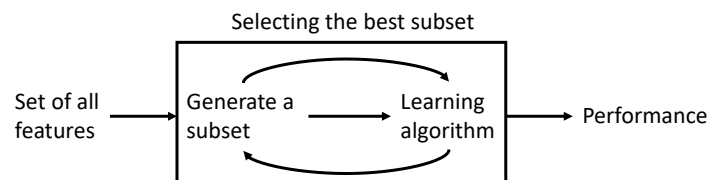
11

Models of Feature Selection

- Filter Model



- Wrapper Model



12

Stepwise Forward Selection

- A wrapper method
- Given a feature set

$$\mathcal{F} = \langle F_1, \dots, F_k \rangle$$

and a data set $\mathcal{D}$, begin with an empty feature set $\mathcal{F}_0 = \langle \cdot \rangle$.

- The process involves selecting one feature at a time, evaluating a model trained with the new feature, retaining the best single feature, and repeating the process.
- Similar in principle to “condensed nearest neighbor.”

Stepwise Forward Selection

Algorithm 10.1 Stepwise Forward Selection

```

1: function SFS( $\mathcal{F}$ ,  $\mathcal{D}_{train}$ ,  $\mathcal{D}_{valid}$ , Learn())
2:    $\mathcal{F}_0 \leftarrow \langle \rangle$ 
3:    $basePerf \leftarrow -\infty$ 
4:   repeat
5:      $bestPerf \leftarrow -\infty$ 
6:     for all  $F \in \mathcal{F}$  do
7:        $\mathcal{F}_0 \leftarrow \mathcal{F}_0 + F$ 
8:        $h \leftarrow \text{Learn}(\mathcal{F}_0, \mathcal{D}_{train})$ 
9:        $currPerf \leftarrow \text{Perf}(h, \mathcal{D}_{valid})$ 
10:      if  $currPerf > bestPerf$  then
11:         $bestPerf \leftarrow currPerf$ 
12:         $bestF \leftarrow F$ 
13:      end if
14:       $\mathcal{F}_0 \leftarrow \mathcal{F}_0 - F$ 
15:    end for
16:    if  $bestPerf > basePerf$  then
17:       $basePerf \leftarrow bestPerf$ 
18:       $\mathcal{F} \leftarrow \mathcal{F} - bestF$ 
19:       $\mathcal{F}_0 \leftarrow \mathcal{F}_0 + bestF$ 
20:    else
21:      exit
22:    end if
23:  until  $\mathcal{F} \leftarrow \langle \rangle$ 
24:  return  $\mathcal{F}_0$ 
25: end function

```

Stepwise Backward Elimination

- A wrapper method
- Given a feature set

$$\mathcal{F} = \langle F_1, \dots, F_k \rangle$$

and a data set $\mathcal{D}$, begin with an empty feature set $\mathcal{F}_0 = \langle F_1, \dots, F_k \rangle$.

- The process involves selecting one feature at a time, evaluating a model trained with the feature removed, removing the worst feature, and repeating the process.
- Similar in principle to “edited nearest neighbor.”

15

Stepwise Backward Elimination

Algorithm 10.2 Stepwise Backward Elimination

```

1: function SBE( $\mathcal{F}$ ,  $\mathcal{D}_{train}$ ,  $\mathcal{D}_{valid}$ , Learn())
2:    $\mathcal{F}_0 \leftarrow \mathcal{F}$ 
3:    $basePerf \leftarrow -\infty$ 
4:   repeat
5:      $bestPerf \leftarrow -\infty$ 
6:     for all  $F \in \mathcal{F}$  do
7:        $\mathcal{F}_0 \leftarrow \mathcal{F}_0 - F$ 
8:        $h \leftarrow \text{Learn}(\mathcal{F}_0, \mathcal{D}_{train})$ 
9:        $currPerf \leftarrow \text{Perf}(h, \mathcal{D}_{valid})$ 
10:      if  $currPerf > bestPerf$  then
11:         $bestPerf \leftarrow currPerf$ 
12:         $bestF \leftarrow F$ 
13:      end if
14:     $\mathcal{F}_0 \leftarrow \mathcal{F}_0 + bestF$ 
15:    end for
16:    if  $bestPerf > basePerf$  then
17:       $basePerf \leftarrow bestPerf$ 
18:       $\mathcal{F} \leftarrow \mathcal{F} - bestF$ 
19:       $\mathcal{F}_0 \leftarrow \mathcal{F}_0 - bestF$ 
20:    else
21:      exit
22:    end if
23:  until  $|\mathcal{F}| = 1$ 
24:  return  $\mathcal{F}_0$ 
25: end function

```

16

Local Search Methods

- Consider searching over the full space of possible features, rather than adding or eliminating features one at a time in a Greedy fashion.
- Referred to as “local search” since it makes local modifications to a complete solution.
- A powerful search method that may have been encountered in
 - Artificial Intelligence
 - Stochastic Search/Optimization
- We address three methods:
 - Hill Climbing
 - Simulated Annealing
 - Genetic Algorithms

Hill Climbing

- Simple Hill Climbing
 - Establish a starting state for search (e.g., a random binary vector of size equal to the number of features, where “1” indicates the feature is selected and “0” indicates the feature is not selected).
 - Evaluate performance by training a model with the selected features.
 - Enter search loop: Incrementally flip bit in the vector, keeping track of performance, retaining the best bit flip.
 - Note: The two stepwise methods are hill climbers.
- Stochastic Hill Climbing (aka first-choice HC or random mutation HC)
 - Establish a starting state for search.
 - Evaluate performance by training a model with the selected features.
 - Enter search loop: Randomly select and flip bit in the vector. If better, take it. If not, discard and try again.

Simulated Annealing

```

function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
           schedule, a mapping from time to "temperature"
  local variables: current, a node
                    next, a node
                    T, a "temperature" controlling prob. of downward steps

  current ← MAKE-NODE(INITIAL-STATE[problem])
  for t ← 1 to ∞ do
    T ← schedule[t]
    if T = 0 then return current
    next ← a randomly selected successor of current
     $\Delta E \leftarrow \text{VALUE}[\textit{next}] - \text{VALUE}[\textit{current}]$ 
    if  $\Delta E > 0$  then current ← next
    else current ← next only with probability  $e^{\Delta E/T}$ 

```



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

19

Genetic Algorithm

Suppose

$$\mathcal{F} = \langle b_0 \ b_1 \ b_2 \ b_3 \ b_4 \ b_5 \ b_6 \ b_7 \ b_8 \ b_9 \rangle$$

$$\mathcal{F}_0 = \langle 1001011010 \rangle$$

$$\mathcal{P}(t) = \{\mathcal{F}_0, \dots, \mathcal{F}_{n-1}\}$$

```

1: function GENETICALGORITHM
2:   t ← 0
3:   initialize(P(t))
4:   f(t) ← evaluateFitness(P(t))
5:   while not(terminatep()) do
6:     t ← t + 1
7:     C'(t) ← select(P(t - 1))
8:     C'(t) ← recombine(C'(t))
9:     C''(t) ← mutate(C'(t))
10:    f(t) ← evaluateFitness(C''(t))
11:    P(t) ← replace(P(t - 1), C''(t), f(t))
12:  end while
13:  return P(t)
14: end function

```



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

20

Selection and Replacement

- Assume “steady state” replacement.
- This means only a small subset of the population is replaced in each generation, rather than the entire population.
- Several methods for selection of parents.
- We apply “fitness proportionate selection”

$$P(X_i) = \frac{f(X_i)}{\sum_{j=0}^{n-1} f(X_j)}$$

- Fitness is based on performance of the model trained with the subset of features.

21

Recombination/Crossover

- A method for mating two (or more) parents selected from the population.
- We assume two parents and generate two offspring.
- Parents

[1 1 1 1 1 1 1 1 1 1]

[0 0 0 0 0 0 0 0 0 0]

- Offspring

[1 1 1 1 0 0 0 0 0 0]

[0 0 0 0 1 1 1 1 1 1]

22

Mutation

- Introduce random variation.
- “March” down the individuals and randomly swap a subset of bits.
- Offspring

[1 1 1 1 0 0 0 0 0 0]

[0 0 0 0 1 1 1 1 1 1]

- Mutated

[1 1 0 1 0 0 0 0 1 0]

[0 0 0 1 1 1 0 1 1 0]

Principal Component Analysis

- Find an orthogonal basis set that explains the variance in the data.
- Let $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ be a set of examples in d -dimensional space (i.e., $\mathbf{x}_i = \langle F_1, \dots, F_d \rangle$).
- Calculate the sample mean: $\mathbf{m} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$.
- Convert to zero-mean: $\mathbf{x}'_i = \mathbf{x}_i - \mathbf{m}$.
- Calculate the variance of the features: $\mathbf{v} = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \mathbf{m})^2 = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}'_i)^2$.

Principal Component Analysis

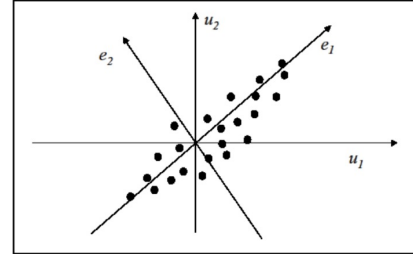
- Consider the *covariance* between pairs of features F_a and F_b .

$$\mathbf{v}_{ab} = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_{ai} - \mathbf{m})^\top (\mathbf{x}_{bi} - \mathbf{m}) = \frac{1}{n} \sum_{i=1}^n \mathbf{x}'_{ai} \mathbf{x}'_{bi}$$

- Apply SVD to the covariance matrix and solve for the eigenvalue matrix.

$$\mathbf{\Lambda} = \mathbf{U}^\top \text{cov}(\mathbf{X}) \mathbf{U}$$

- The off-diagonal entries of $\mathbf{\Lambda}$ are all zero.
- The diagonal entries of $\mathbf{\Lambda}$ are the square roots of the eigenvalues of $\text{cov}(\mathbf{X})$, which are the variances of the transformed data along the principal axes (components).
- The row vectors of $\mathbf{U}$ are the eigenvectors of $\text{cov}(\mathbf{X})$ and are called the “principal components” of $\text{cov}(\mathbf{X})$.
- Choose $k < d$, select the k eigenvectors with largest eigenvalues, and project data onto the the eigenvectors, $\mathbf{p} = \mathbf{u}_i^\top \mathbf{x}'$.



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

25

Determining k

- Sort the eigenvalues λ_i^2 (recalling the singular values are the square root of the eigenvalues).
- Calculate the “proportion of variance”:

$$PoV = \frac{\sum_{i=1}^k \lambda_i^2}{\sum_{j=1}^d \lambda_j^2}.$$

- Select k such that PoV exceeds some threshold (i.e., 90%).

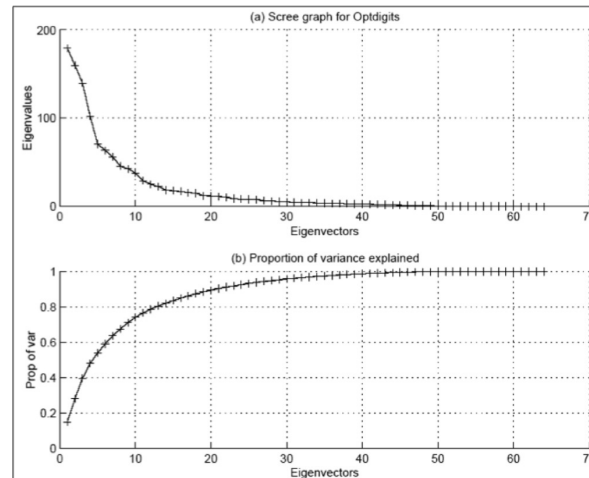


MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

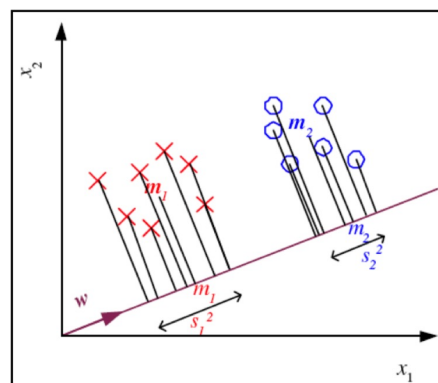
26

Scree Graph



27

Fisher's Linear Discriminant Analysis



28

Finding the Linear Discriminant

- Let $\mathcal{F} = \langle F_1, \dots, F_d \rangle$ be a feature vector.
- Let $\mathbf{m}_1$ be the mean vector of examples from class C_1 and $\mathbf{m}_2$ be the mean vector of examples from class C_2 (assuming two classes).
- Let $\mathbf{m}_1^p$ be the mean vector after projection (similarly for $\mathbf{m}_2^p$).
- Given a sample of data $\mathcal{D} = \{\mathbf{x}^t, c^t\}$.

$$\mathbf{m}_1^p = \frac{\sum_t \mathbf{w}^\top \mathbf{x}^t c^t}{\sum_t c^t} = \mathbf{w}^\top \mathbf{m}_1 \Rightarrow s_1^2 = \sum_t (\mathbf{w}^\top \mathbf{x} - \mathbf{m}_1^p)^2 c^t$$

$$\mathbf{m}_2^p = \frac{\sum_t \mathbf{w}^\top \mathbf{x}^t (1 - c^t)}{\sum_t (1 - c^t)} = \mathbf{w}^\top \mathbf{m}_2 \Rightarrow s_2^2 = \sum_t (\mathbf{w}^\top \mathbf{x} - \mathbf{m}_2^p)^2 (1 - c^t)$$

Following Projection

- We want the two classes to be well separated.
 $|\mathbf{m}_1^p - \mathbf{m}_2^p|$ large
- We also want the two classes to have minimal scatter.
 $s_1^2 + s_2^2$ small
- Find the weight vector $\mathbf{w}$ such that

$$J(\mathbf{w}) = \frac{(\mathbf{m}_1^p - \mathbf{m}_2^p)^2}{s_1^2 + s_2^2}$$

Between-Class Scatter

Rewrite the numerator in terms of a projection onto $\mathbf{w}$.

$$\begin{aligned} (\mathbf{m}_1^p - \mathbf{m}_2^p)^2 &= (\mathbf{w}^\top \mathbf{m}_1 - \mathbf{w}^\top \mathbf{m}_2)^2 \\ &= \mathbf{w}^\top (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^\top \mathbf{w} \\ &= \mathbf{w}^\top \mathbf{S}_B \mathbf{w} \end{aligned}$$

where

$$\mathbf{S}_B = (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^\top$$

Within Scatter

Rewrite the denominator in terms of a projection onto $\mathbf{w}$.

$$\begin{aligned} s_1^2 &= \sum_t (\mathbf{w}^\top \mathbf{x}^t - \mathbf{m}_1^p)^2 c^t \\ &= \sum_t \mathbf{w}^\top (\mathbf{x}^t - \mathbf{m}_1)(\mathbf{x}^t - \mathbf{m}_1)^\top \mathbf{w} c^t \\ &= \mathbf{w}^\top \mathbf{S}_1 \mathbf{w} \end{aligned}$$

where

$$\mathbf{S}_1 = \sum_t c^t (\mathbf{x}^t - \mathbf{m}_1)(\mathbf{x}^t - \mathbf{m}_1)^\top$$

Total class scatter is $\mathbf{S}_W = \mathbf{S}_1 + \mathbf{S}_2$, and denominator becomes $s_1^2 + s_2^2 = \mathbf{w}^\top \mathbf{S}_W \mathbf{w}$.

Final Objective Function

- The final objective function to optimize then becomes

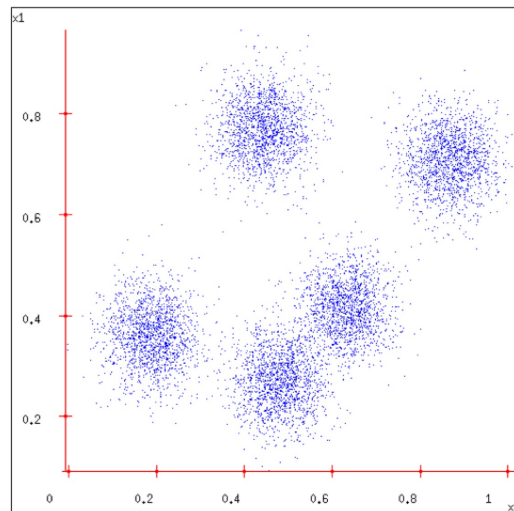
$$J(\mathbf{w}) = \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}.$$

- Taking the derivative, setting to zero, and solving yields

$$\begin{aligned} \nabla_{\mathbf{w}} J(\mathbf{w}) &= 0 \\ \mathbf{w} &= \mathbf{S}_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2). \end{aligned}$$

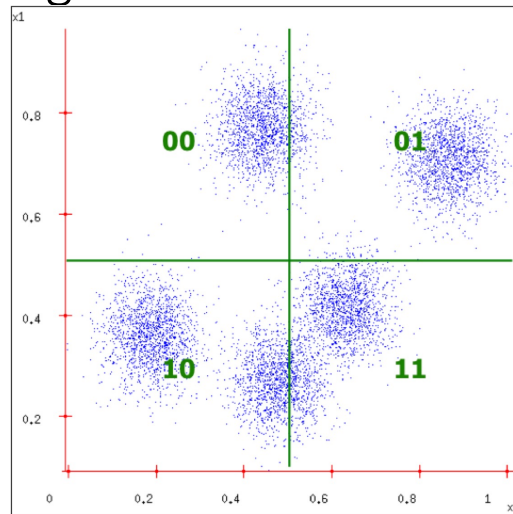
33

Clustering Data



34

Grid Clustering



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

35

Expectation Maximization

- A local search method to find values of variables that are not observed directly.
- Employs a maximum-likelihood approach.
- Assume we have observed variables $\mathcal{X}$ and unobserved variables $\mathcal{U}$.
- The goal is to find the set of parameters Φ that maximizes the likelihood function $\mathcal{L}(\Phi|\mathcal{X})$.
- When there are hidden (i.e., latent) factors that affect this, we model them with $\mathcal{U}$ and then seek to maximize $\mathcal{L}(\Phi|\mathcal{X}, \mathcal{U})$.
- We will use this idea to cluster the data.



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

36

EM for Clustering

- Bivariate Gaussians with equal variance
- k -means clustering
- **Assumptions**
 - We have k multi-variate Gaussian/spherical clusters.
 - Each cluster has an unknown mean vector: $\langle \mu_1, \dots, \mu_k \rangle$.
 - We do not know which Gaussian generated which data point.
- We can apply EM to find the cluster means.
- We can use the cluster means to assign the points.

k -Means Clustering

Example

- Assume we have two clusters.
- Describe a generated instance as $y_i = \langle x_i, u_{i1}, u_{i2} \rangle$, where $u_{ij} = 1$ means x_i was generated by cluster j .
- Begin by guessing values for $h = \langle \mu_1, \mu_2 \rangle$ at random.

E Step: (Expectation)

$$E[u_{ij}] = \frac{P(x_i | \mu_j)}{\sum_{k=1}^2 P(x_i | \mu_k)} = \frac{\exp\left[-\frac{1}{2\sigma^2}(x_i - \mu_j)^2\right]}{\sum_{k=1}^2 \exp\left[-\frac{1}{2\sigma^2}(x_i - \mu_k)^2\right]}$$

M Step: (Maximization, $h' = \langle \mu'_1, \mu'_2 \rangle$)

$$\mu_j = \frac{\sum_{i=1}^m E[u_{ij}]x_i}{\sum_{i=1}^m E[u_{ij}]}$$

k -Means Clustering

Algorithm 10.3 K -Means Clustering

```

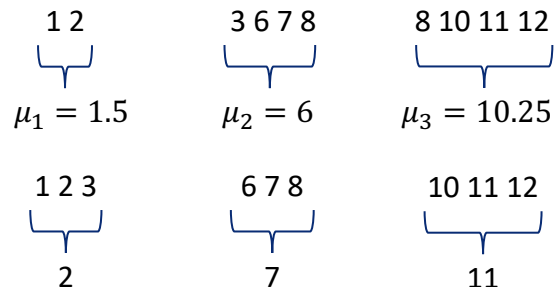
1: function KMEANS( $\mathcal{D}, k$ )
2:   initialize  $\mu_1, \dots, \mu_k$  randomly
3:   repeat
4:     for all  $\mathbf{x}_i \in \mathcal{D}$  do
5:        $c \leftarrow \arg \min_{\mu_j} d(\mathbf{x}_i, \mu_j)$             $\triangleright d()$  is the distance between  $\mathbf{x}_i$  and  $\mu_j$ .
6:       assign  $\mathbf{x}_i$  to the cluster  $c$ 
7:     end for
8:     recalculate all  $\mu_j$  based on new clusters
9:   until no change in  $\mu_1, \dots, \mu_k$ 
10:  return  $\mu_1, \dots, \mu_k$ 
11: end function

```

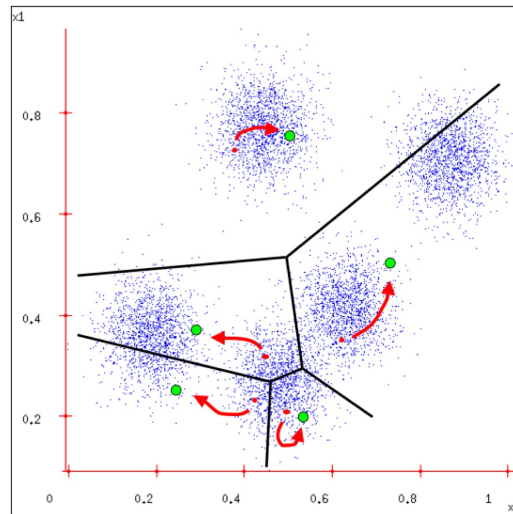
An Illustration of k -Means

1 2 3 6 7 8 10 11 12

$K = 3, \mu_1 = 1, \mu_2 = 4, \mu_3 = 12$



Another Illustration of k -Means



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

41

Improved Initialization: k -means++

- Shortcomings of k -means
 - Worst-case time complexity is super-polynomial (note that both k -means and k -medoids are actually NP-hard).
 - Final clusters can be objectively bad relative to optimal clustering (i.e., unbounded relative to optimal clustering).
- Both issues are affected by the initial cluster centroids, but especially the second.
- Extension to k -means, known as k -means++ focuses on the initial centroids.
- The result guarantees clusters whose suboptimality can be bounded.



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

42

The k -means++ Algorithm

- Initialize $\mathbf{C} = \emptyset$.
- Choose an initial cluster center, $\mathbf{c}_1$, uniformly at random from $\mathcal{X}$ and add to $\mathbf{C}$. Thus, the initial center is an example in the data set.
- Repeat Let $\delta(\mathbf{x})$ denote the minimum distance from $\mathbf{x}$ to the closest cluster center already chosen.

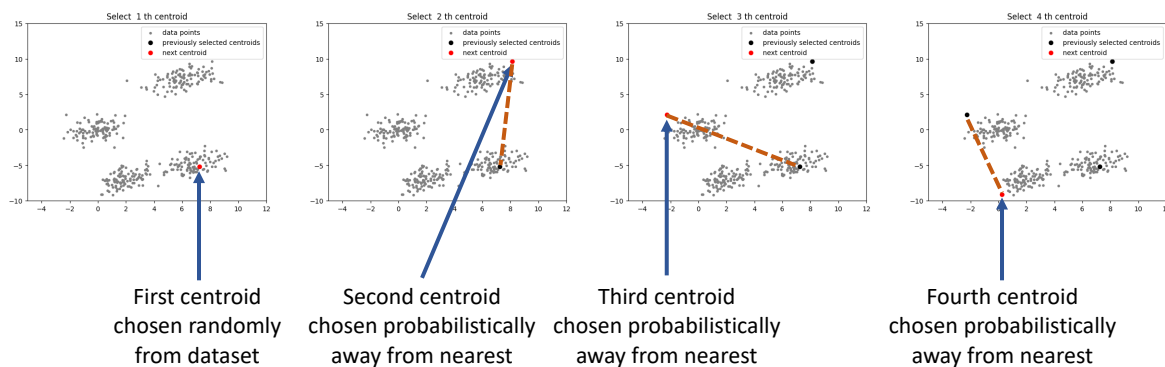
Choose the next center, $\mathbf{c}_i$, such that $\mathbf{c}_i = \mathbf{x}' \in \mathcal{X} \setminus \mathbf{C}$ with probability,

$$\frac{\delta(\mathbf{x}')}{\sum_{\mathbf{x} \in \mathcal{X} \setminus \mathbf{C}} \delta(\mathbf{x})}$$

until k cluster centers have been selected.

43

Example



44

Theory

- Let $\phi = \sum_{\mathbf{x} \in \mathcal{X}} \min_{\mathbf{c} \in \mathcal{C}} \|\mathbf{x} - \mathbf{c}\|^2$ denote the “potential” of clusters $\mathcal{C}$.
- Let ϕ_{opt} denote the “potential” of the optimal clustering of $\mathcal{X}$.
- **Theorem:** If $\mathcal{C}$ is constructed using k -means++, then $E[\phi] \leq 8(\ln k + 2)\phi_{\text{opt}}$.
- We describe this theorem as saying that k -means++ guarantees that the resulting clustering will be $O(\log k)$ -competitive relative to the optimal clustering.